

# Control and State Estimation of the Pendubot

## ECE 238 Final Project

Heerthan Shanthan and Yang Chen

University of California, Santa Barbara

**Abstract.** This project focuses on control and state estimation of the Pendubot, an underactuated system commonly used for nonlinear control research. The Pendubot is a two-link robot with a single actuator at its shoulder joint, leaving the elbow joint passive. This work considers stabilizing two different equilibria using Linear Quadratic Regulator (LQR) applied to linearized models of the system. Because only joint angles are measured and both process and measurement noise are present, full-state estimation is performed using Kalman filtering. Depending on the objective, we use a standard Kalman Filter (KF), or an Extended Kalman Filter (EKF). The goal of this project is to simulate the proposed control and estimation methods in MATLAB/Simulink, and evaluate their performance.

## 1 Introduction

The Pendubot is a two-link, underactuated robotic system, meaning that only one of its two degrees of freedom (DOFs) is actuated. A single motor torque is applied at the first joint, while the second joint is passive. As a nonlinear, underactuated system, the Pendubot has been widely studied, and numerous control strategies have been proposed in [1, 5, 6]. Similar to the approach in [5], we employ LQR to design our control law. For stabilization around the downward equilibrium, a single LQR controller is sufficient, and a linear KF provides accurate state estimation. Stabilization around the upward equilibrium, however, requires a different strategy. The system must first be swung up to a region where the dynamics can be linearized about the upright equilibrium, at which point we switch to a second LQR controller. For the swing-up phase, we generate a reference trajectory using trajectory optimization and linearize the dynamics along this trajectory. Due to the strong nonlinearities present during this phase, an EKF is necessary to get accurate state estimates for the upright stabilization task.

## 2 System Dynamics

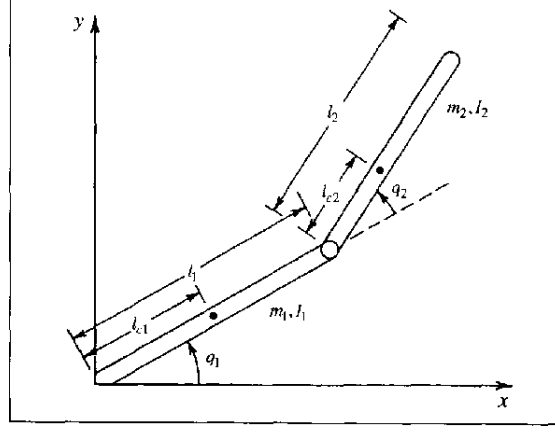


Fig. 1: The Pendubot

The two degrees of freedom (DOFs) of the Pendubot are shown in Fig. 1, where  $q_1$  denotes the collocated (actuated) joint angle of the first link and  $q_2$  denotes the unactuated joint angle of the second link. The angular velocities and accelerations are defined as  $\dot{q} = [\dot{q}_1 \ \dot{q}_2]^T$  and  $\ddot{q} = [\ddot{q}_1 \ \ddot{q}_2]^T$ . We define the state vector as  $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]^T$  and the control input torque as  $\tau$ . Using the Euler–Lagrange equation [6], the dynamics of the Pendubot are described as

$$d_{11}\ddot{q}_1 + d_{12}\ddot{q}_2 + h_1 + \phi_1 = \tau \quad (1)$$

$$d_{21}\ddot{q}_1 + d_{22}\ddot{q}_2 + h_2 + \phi_2 = 0 \quad (2)$$

where  $\tau$  is the motor torque applied at the joint angle of the first link, and

$$\begin{aligned} d_{11} &= \theta_1 + \theta_2 + 2\theta_3 \cos(q_2), & d_{22} &= \theta_2, \\ d_{12} &= d_{21} = \theta_2 + \theta_3 \cos(q_2), & h_2 &= \theta_3 \sin(q_2) \dot{q}_1^2, \\ h_1 &= -\theta_3 \sin(q_2) \dot{q}_2^2 - 2\theta_3 \sin(q_2) \dot{q}_1 \dot{q}_2, & \phi_1 &= \theta_4 g \cos(q_1) + \theta_5 g \cos(q_1 + q_2), \\ \phi_2 &= \theta_5 g \cos(q_1 + q_2) \end{aligned} \quad (3)$$

where

$$\begin{aligned} \theta_1 &= m_1 \ell_{c1}^2 + m_2 \ell_1^2 + I_1 = 0.0308 \text{ Vs}^2, & \theta_4 &= m_1 \ell_{c1} + m_2 \ell_1 = 0.2087 \text{ Vs}^2/\text{m}, \\ \theta_2 &= m_2 \ell_{c2}^2 + I_2 = 0.0160 \text{ Vs}^2, & \theta_5 &= m_2 \ell_{c2} = 0.0630 \text{ Vs}^2, \\ \theta_3 &= m_2 \ell_1 \ell_{c2} = 0.0095 \text{ Vs}^2. \end{aligned} \quad (4)$$

through parameter identification of the Pendubot in the UCSB Controls Lab [4], and  $g = 9.8m/s^2$ .

Setting up (1) and (2) in matrix form gives

$$\begin{bmatrix} d_{11} & d_{12} \\ d_{21} & d_{22} \end{bmatrix} \ddot{q} + \begin{bmatrix} h_1 + \phi_1 \\ h_2 + \phi_2 \end{bmatrix} = \begin{bmatrix} \tau \\ 0 \end{bmatrix} \quad (5)$$

Solving directly for  $\ddot{q}$  from (5) allows us to define the nonlinear state function  $f(x, u) = \dot{x}$ . The Pendubot in the lab outputs only  $q_1$  and  $q_2$  through encoders, meaning the angular velocities must be estimated.

### 3 Control Law

Our goal for this section is to design a control feedback law  $u = -Kx$  that stabilizes the system about a reference trajectory or equilibrium point. We use discrete-time LQR, an optimal control method that places the system closed-loop poles by minimizing a quadratic cost function. Because LQR requires linear dynamics, the nonlinear system is linearized about a nominal trajectory or equilibrium. The resulting  $A$  and  $B$  matrices describe the error dynamics between the true and nominal states.

The LQR cost function is

$$J = \sum_{k=0}^{N-1} \left( (x(k) - x_{\text{nom}}(k))^T Q (x(k) - x_{\text{nom}}(k)) + (u(k) - u_{\text{nom}}(k))^T R (u(k) - u_{\text{nom}}(k)) \right), \quad (6)$$

subject to the linearized discrete-time error dynamics

$$x(k+1) - x_{\text{nom}}(k+1) = A_d(k) (x(k) - x_{\text{nom}}(k)) + B_d(k) (u(k) - u_{\text{nom}}(k)). \quad (7)$$

where  $A_d$  and  $B_d$  are the Euler-discretized  $A$  and  $B$  matrices. The optimal control input that minimizes the cost is

$$u(k) = u_{\text{nom}}(k) - K(k) (x(k) - x_{\text{nom}}(k)). \quad (8)$$

Full derivations are omitted for brevity and can be found in [7].

#### 3.1 Control Law for Equilibrium 1

The first equilibrium of interest is the downward hanging position,  $x_{eq1} = [-\pi/2, 0, 0, 0]^T$ . Linearizing the dynamics about the  $x_{eq1}$  gives the following discrete-time state-space model:

$$A = \begin{bmatrix} 1 & 0 & 0.001 & 0 \\ 0 & 1 & 0 & 0.001 \\ -0.0669 & 0.0248 & 1 & 0 \\ 0.0687 & -0.1053 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 0.0449 \\ -0.0851 \end{bmatrix}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}, \quad D = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Both the controllability and observability matrices are full rank, indicating that the system is both controllable and observable at this equilibrium.

Using discrete-time LQR, we get the feedback gain

$$K = [-0.5475 \ -0.6718 \ 0.2223 \ -0.07]$$

### 3.2 Control Law for Equilibrium 2

The second equilibrium of interest is  $x_{eq2} = [\pi/2 \ 0 \ 0 \ 0]^T$ . Unlike  $x_{eq1}$ ,  $x_{eq2}$  requires a swing-up before the system reaches a linearizable region. To accomplish this, we compute a reference trajectory for the system to track during swing-up.

**Reference Trajectory Generation** The reference trajectory is generated using open-loop trajectory optimization. Using OptimTraj [3], the continuous-time optimal control problem is transcribed into a discrete-time non-linear program by discretizing the state and control trajectories at specific points in time called collocation points. The integrals of the dynamics and cost function are approximated using trapezoidal quadrature, while dynamic constraints are approximated in between collocation points. [2] explains this in further detail.

For a set of collocation points  $k = 0, \dots, N$  and  $h_k = (t_{k+1} - t_k)$ , the problem becomes:

$$\begin{aligned} \min_{\{x_k, u_k\}} \quad & J = \sum_{k=0}^{N-1} h_k \\ \text{s.t.} \quad & x_{k+1} - x_k - \frac{h_k}{2} \left( f(x_k, u_k) + f(x_{k+1}, u_{k+1}) \right) = 0, \quad k = 0, \dots, N-1, \\ & x_0 = x_{ic}, \quad x_N = x_{tf}, \\ & x_{\min} \leq x_k \leq x_{\max}, \quad k = 0, \dots, N, \\ & u_{\min} \leq u_k \leq u_{\max}, \quad k = 0, \dots, N. \end{aligned}$$

where  $x_{ic} = x_{eq1}$ ,  $x_{tf} = x_{eq2}$ ,  $x_{\max} = [2\pi \ 2\pi \ \infty \ \infty]$ ,  $x_{\min} = -x_{\max}$ ,  $u_{\max} = 10$ ,  $u_{\min} = -u_{\max}$ , and  $t_f = 2$ .

The approximated cost function integral is chosen to minimize the total time of the trajectory. However, since we fix  $t_f$ , this objective becomes irrelevant. Because our only goal is to generate a dynamically feasible trajectory, the cost function serves as a placeholder, allowing the solver to generate a valid solution quickly.

Figure 2 shows the generated reference trajectories and the corresponding optimal control input over time from trajectory optimization.

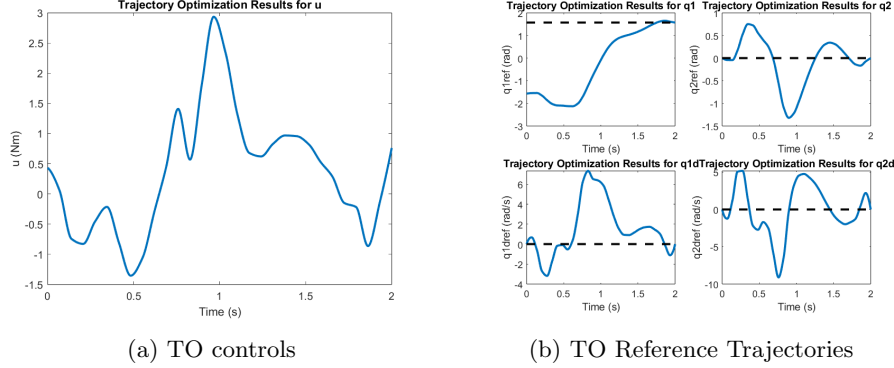


Fig. 2: TO Results

At each time step during swing-up, the system is linearized about the reference trajectory, resulting in a time-varying feedback gain matrix  $K(t)$ . Once the system enters the basin of attraction of  $x_{eq2}$ , we switch to a second LQR controller designed solely for stabilization about  $x_{eq2}$ . Here, we note that  $K$  and linearized matrices are constant

While results are omitted for brevity, the observability and controllability corresponding to the state space matrices maintain full rank throughout the entire trajectory, making the system controllable and observable.

## 4 State Estimation

Since the system is observable at both equilibrium points, we can construct a Kalman filter to estimate the full state. The main role of the Kalman filter is to accurately estimate states in the presence of process and measurement noise. Revisiting the linearized system (7), we augment it with additive noise and include the output equation:

$$\begin{aligned} x(k+1) &= x_{\text{nom}}(k+1) + A_d(k) (x(k) - x_{\text{nom}}(k)) + B_d(k) (u(k) - u_{\text{nom}}(k)) + w(k), \\ y(k) &= C x(k) + v(k). \end{aligned} \tag{9}$$

where  $w(k)$  is the process noise vector and  $v(k)$  is the measurement noise. The noise covariance matrices are assumed stationary over time and given by

$$\begin{aligned} Q_N &= \mathbb{E}[w(k)w(k)^\top], \\ R_N &= \mathbb{E}[v(k)v(k)^\top]. \end{aligned} \tag{10}$$

We chose to use a linear Kalman Filter for stabilization of  $x_{eq1}$ , while an extended Kalman Filter was needed to stabilize the swing-up reference trajectory and  $x_{eq2}$ .

### 4.1 Linear Kalman Filter

The discrete-time Linear Kalman Filter outputs the state estimate  $x_{est}$  and error covariance matrix  $P$ . At each timestep,  $x_{est}$  can be computed recursively through

a prediction step and a correction step. The  $A$  and  $B$  matrices are constant and linearized about  $x_{eq1}$ , so that they represent the error dynamics of the system. Therefore, it is necessary to add the equilibrium back after propagating the error state. The KF update equations are

$$\begin{aligned} x_{\text{pred}} &= A(x_{\text{est}}^- - x_{\text{eq}}) + x_{\text{eq}} + Bu, \\ P_{\text{pred}} &= APA^\top + Q_N, \\ L &= P_{\text{pred}}C^\top(CP_{\text{pred}}C^\top + R_N)^{-1}, \\ x_{\text{est}}^+ &= x_{\text{pred}} + L(y^+ - Cx_{\text{pred}}), \\ P &= (I - LC)P_{\text{pred}}(I - LC)^\top + LRL^\top. \end{aligned} \quad (11)$$

where  $L$  is the Kalman correction gain,  $Q_N$  is the process noise covariance matrix, and  $R_N$  is the measurement noise covariance matrix.

## 4.2 Extended Kalman Filter (EKF)

The high nonlinearity of the system during swing-up causes the KF to produce poor state estimates. This motivated the use of an EKF, which outputs the state estimates  $x_{\text{est}}$  and error covariance matrix  $P$ . At each timestep,  $x_{\text{est}}$  is updated recursively through a prediction step and a correction step. The EKF update equations are

$$\begin{aligned} x_{\text{pred}} &= f(x_{\text{est}}^-, u^+), \\ P_{\text{pred}} &= APA^\top + Q_N, \\ L &= P_{\text{pred}}C^\top(CP_{\text{pred}}C^\top + R_N)^{-1}, \\ x_{\text{est}}^+ &= x_{\text{pred}} + L(y^+ - Cx_{\text{pred}}), \\ P &= (I - LC)P_{\text{pred}}(I - LC)^\top + LRL^\top. \end{aligned} \quad (12)$$

Although similar in structure to the KF, the EKF uses the full nonlinear model for state prediction, while  $A$  is linearized about the current operating point. This makes the EKF better suited for handling the nonlinear behavior.

## 5 Results & Simulation

We present simulation results for one scenario corresponding to each of the two equilibria. All simulations were conducted in MATLAB/Simulink using a fixed sampling interval  $\Delta t = 0.001$ .

### 5.1 Equilibrium 1

For stabilization about  $x_{eq1}$ , the system was simulated from the initial condition  $x_0 = [-\pi/4 \ 0 \ 0 \ 0]^\top$ .

The LQR weighting matrices were selected as

$$Q = \text{diag}[15 \ 15 \ 1 \ 1] \quad R = 50$$

and the noise covariance matrices as:

$$Q_N = \text{diag} [1e-9 \ 1e-9 \ 1e-9 \ 1e-9] \quad R_N = \text{diag} [0.001 \ 0.001]$$

The simulation was run for  $10s$ , during which the system converged to  $x_{tf} = [-1.5708 \ 0.001 \ 0.007 \ -0.005]^T$ , demonstrating successful stabilization of  $x_{eq1}$ .

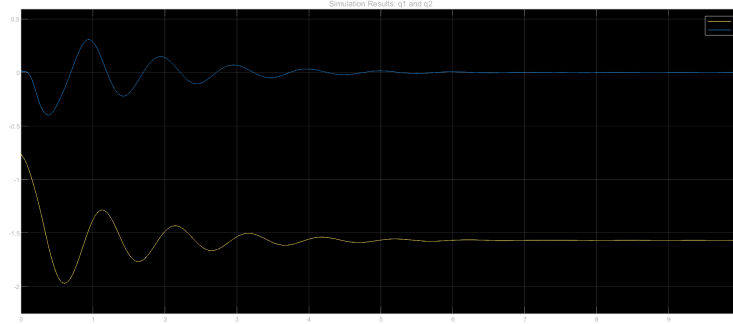


Fig. 3: Simulation Results:  $q_1$  and  $q_2$

The Linear Kalman Filter accurately estimated all four states. Using the true simulated dynamics as ground truth, Figure 4 shows that unmeasured joint velocities are observed with a high accuracy.

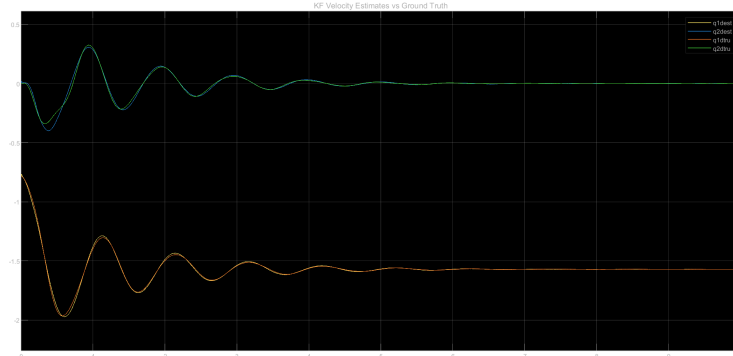
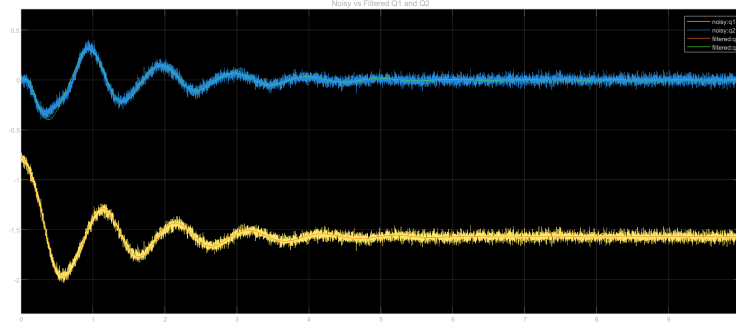


Fig. 4: Estimate vs Ground Truth:  $\dot{q}_1$  and  $\dot{q}_2$

The filter also effectively filters out measurement noise, as seen in the Figure 5 for  $q_1$  and  $q_2$ .

Fig. 5: Noisy vs Filtered:  $q_1$  and  $q_2$ 

## 5.2 Equilibrium 2

For swing-up and stabilization about  $x_{eq2}$ , the system was simulated from  $x_0 = x_{eq1}$ . The LQR weighting matrices for both the trajectory-tracking controller and the final stabilizing controller were chosen as

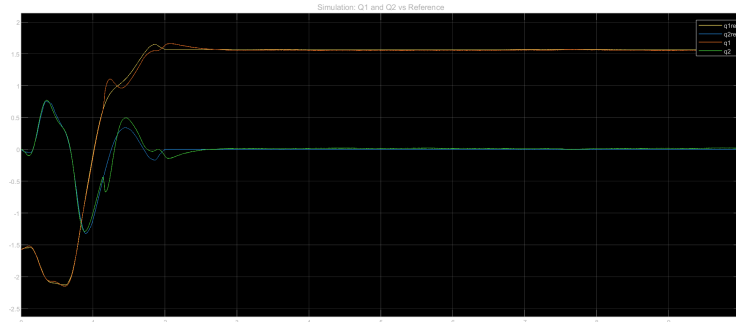
$$Q = \text{diag} [150 \ 250 \ 8 \ 10] \quad R = 150 \quad (13)$$

with noise covariances

$$Q_N = \text{diag} [1e-9 \ 1e-9 \ 1e-9 \ 1e-9] \quad R_N = \text{diag} [1e-6 \ 1e-6] \quad (14)$$

We switched from the time-varying LQR controller to the equilibrium LQR controller when  $\|q - q_{eq}\|_2 \leq 0.3$ .

The simulation was run for 10s, and the system converged to  $x_{tf} = [1.55 \ 0.02 \ 0.009 \ -0.009]^T$ , demonstrating successful swing-up trajectory tracking and stabilization about  $x_{eq2}$ .

Fig. 6:  $q_1$  and  $q_2$  vs Reference Trajectory

Because the EKF had difficulties filtering out measurement noise, we kept the  $R_N$  small and focused on the performance of velocity estimation. As expected



under low noise, the EKF accurately estimates all states.

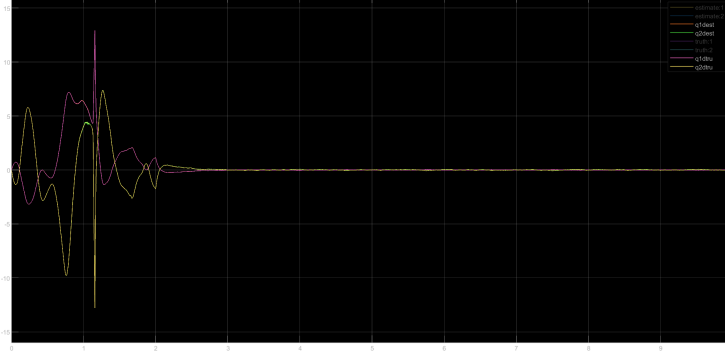


Fig. 7: Estimate vs Ground Truth:  $\dot{q}_1$  and  $\dot{q}_2$

A sharp, brief spike in the joint velocities was observed in both the ground truth dynamics and the EKF estimates. This behaviour is not an artifact of the filter, but rather the system dynamics. We suspect this is related to the corresponding spike in the condition number of the controllability matrix when the system passes near the dynamically sensitive region around  $q_1 = 0$  [6]. Although the controllability matrix remains full rank at all times, it becomes highly ill-conditioned near this region which could indicate weak controllability.

## 6 Conclusion

In this project, we combined Kalman filtering with LQR control to achieve stabilization around two equilibria of the Pendubot. For the upright equilibrium, we first used trajectory optimization to compute a reference swing-up trajectory, after which a time-varying LQR controller stabilized the system. An EKF was used for state estimation during the upright swingup and stabilization, while a standard KF was sufficient for stabilization around the downward equilibrium. Our simulation results, based on experimentally identified model parameters, exhibit stable system responses which stabilizes both of the desired equilibrium in the presence of noise. Due to inaccurate estimates from the EKF for the upright equilibrium at higher magnitudes of noise, future work could involve using an Unscented Kalman Filter (UKF) which could potentially account for the high nonlinearities of the system that arises especially when far away from an equilibrium.

## References

1. Fantoni, I., et.al: Energy based control of the pendubot (2000), <https://ieeexplore.ieee.org/document/847110>

2. Kelly, M.: An introduction to trajectory optimization: How to do your own direct collocation. SIAM Review **59**(4), 849–904 (2017), [https://www.matthexpeterkelly.com/research/MatthewKelly\\_IntroTrajectoryOptimization\\_SIAM\\_Review\\_2017.pdf](https://www.matthexpeterkelly.com/research/MatthewKelly_IntroTrajectoryOptimization_SIAM_Review_2017.pdf)
3. Kelly, M.P.: OptimTraj: Trajectory Optimization for Matlab (12 2022). <https://doi.org/10.5281/zenodo.7430524>, <https://github.com/MatthewPeterKelly/OptimTraj>
4. Mechatronic Systems Inc.: Pendubot’s user manual (1999)
5. Nguyen, D., et.al: Application of lqg control for pendubot system (2024), <https://ejournal.ptti.web.id/index.php/jfsc/article/view/171>
6. Spong, M., Block, D.: The pendubot: a mechatronic system for control research and education (1995)
7. Tedrake, R.: Chapter 8, linear quadratic regulator (2024), <https://underactuated.mit.edu/lqr.html>